

**Syllabus:**

**Binary Search-** Introduction, Applications: Median of two sorted arrays, Find the fixed point in a given array, Find Smallest Common Element in All Rows, Longest Common Prefix, Koko Eating Bananas.

**-Page No: 01**

**Greedy Method:** General method – Applications –Minimum product subset of an array, Best Time to Buy and Sell Stock, Knapsack problem, Minimum cost spanning trees, Single source shortest path problem.

**-Page No: 31**

---

**I. Binary Search****Introduction:**

- Given a sorted array `arr[]` of  $n$  elements, write a function to search a given element  $x$  in `arr[]`.
- A simple approach is to do **Linear Search**.
- The time complexity of above algorithm is  $O(n)$ . Another approach to perform the same task is using Binary Search.
- A binary search or half-interval search algorithm finds the position of a specified value (the input "key") within a sorted array.
- In each step, the algorithm compares the input key value with the key value of the middle element of the array.
- If the keys match, then a matching element has been found so its index, or position, is returned.
- Otherwise, if the sought key is less than the middle element's key, then the algorithm repeats its action on the sub-array to the left of the middle element or, if the input key is greater, on the sub-array to the right.
- If the remaining array to be searched is reduced to zero, then the key cannot be found in the array and a special "Not found" indication is returned.
- Every iteration eliminates half of the remaining possibilities. This makes binary searches very efficient - even for large collections.
- Binary search requires a sorted collection. Also, binary searching can only be applied to a collection that allows random access (indexing).

**Worst case performance:  $O(\log n)$**

**Best case performance:  $O(1)$**

If searching for 23 in the 10-element array:

|                                       |   |   |   |    |    |    |    |    |    |    |
|---------------------------------------|---|---|---|----|----|----|----|----|----|----|
|                                       | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 |
| 23 > 16,<br>take 2 <sup>nd</sup> half | L |   |   |    |    |    |    |    |    | H  |
|                                       | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 |
| 23 < 56,<br>take 1 <sup>st</sup> half |   |   |   |    |    |    |    |    |    |    |
|                                       | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 |
| Found 23,<br>Return 5                 |   |   |   |    |    |    |    |    |    |    |
|                                       | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 56 | 72 | 91 |

### Working:

Binary Search Algorithm can be implemented in two ways which are discussed below.

1. Recursive Method.
2. Iterative Method.

#### 1. Recursive Algorithm ( Divide and Conquer Technique)

**Algorithm BinSrch** (a, low, high, x)

//Given an array a [ low : high ] of elements in increasing

//order,  $1 \leq \text{low} \leq \text{high}$ , determine whether x is present, and

//if so, return j such that  $x = a[j]$ ; else return 0.

```
{
    if( low == high ) then // If small(P)
    {
        if( x = a[low] ) then return low;
        else return 0;
    }
    else
    {
        //Reduce p into a smaller subproblem.
        mid:= (low+high)/2
        if( x = a[mid] ) then return mid;
        else if ( x < a[mid] ) then
            return BinSrch(a, low, mid-1, x);
        else
            return BinSrch(a, mid+1, high, x);
    }
}
```

**2. Iterative Algorithm ( Divide and Conquer Technique)**

Algorithm BinSearch(a, n, x)

// a is an array of size n, x is the key element to be searched.

```
{   low:=1;  high:=n;
  while( low ≤ high)
  {
      mid:=(low+high)/2;
      if (x==a[mid])
      {
          return  mid;
      }
      if( x < a[mid] ) then high := mid-1;
      else low := mid+1;
  }
  return 0;
}
```

**Time complexity of Binary Search**

➤ If the time for diving the list is a constant, then the computing time for binary search is described by the recurrence relation.

**Successful searches:**

|      |          |           |
|------|----------|-----------|
| Best | Average  | Worst     |
| O(1) | O(log n) | O( log n) |

**Unsuccessful searches:**

|          |          |           |
|----------|----------|-----------|
| Best     | Average  | Worst     |
| O(log n) | O(log n) | O( log n) |

**Program for Iterative binary search :**     **BinarySearch\_iterative.java**

```
import java.util.*;
class BinarySearch_iterative
{
    int binarySearch(int array[], int x, int low, int high)
    {

        // Repeat until the pointers low and high meet each other
        while (low <= high)
        {
            int mid = low + (high - low) / 2;

            if (array[mid] == x)
                return mid;

            if (array[mid] < x)
                low = mid + 1;

            else
                high = mid - 1;
        }

        return -1;
    }

    public static void main(String args[])
    {
        BinarySearch_iterative ob = new BinarySearch_iterative ( );

        Scanner sc=new Scanner(System.in);

        System.out.println("enter array size");

        int n = sc.nextInt();

        int array[]=new int[n];

        System.out.println("enter the elements of array ");

        for(int i=0;i<n;i++)
        {
            array[i] =sc.nextInt();
        }
        // Applying sort() method over to above array by passing the array as an argument
        Arrays.sort(array);
    }
}
```

```
// Printing the array after sorting
System.out.println("sorted array:"+ Arrays.toString(array));

System.out.println("Enter the key");
int key=sc.nextInt();

int result = ob.binarySearch(array, key, 0, n - 1);
if (result == -1)
    System.out.println("Not found");
else
    System.out.println("Element found at index " + result);
}
```

**Output:****Case-1:**

```
enter array size
5
enter the elements of array
33
65
32
68
95
sorted array:[32, 33, 65, 68, 95]
Enter the key
59
Not found
```

**Case-2:**

```
enter array size
5
enter the elements of array
33
65
32
68
95
sorted array:[32, 33, 65, 68, 95]
enter array the key
68
Element found at index 3
```

**Program for Recursive binary search:****BinarySearch\_recursive.java**

```
import java.util.*;
class BinarySearch_recursive
{
    int binarySearch(int array[], int x, int low, int high) {

        if (high >= low) {
            int mid = low + (high - low) / 2;

            // If found at mid, then return it
            if (array[mid] == x)
                return mid;

            // Search the left half
            if (array[mid] > x)
                return binarySearch(array, x, low, mid - 1);

            // Search the right half
            return binarySearch(array, x, mid + 1, high);
        }

        return -1;
    }

    public static void main(String args[])
    {
        BinarySearch_recursive ob = new BinarySearch_recursive();

        Scanner sc=new Scanner(System.in);

        System.out.println("enter array size");

        int n = sc.nextInt();

        int array[]=new int[n];

        System.out.println("enter the elements of array ");

        for(int i=0;i<n;i++)
        {
            array[i] =sc.nextInt();
        }
        // Applying sort() method over to above array
        // by passing the array as an argument
```

```
Arrays.sort(array);

// Printing the array after sorting
System.out.println("sorted array[]" + Arrays.toString(array));
System.out.println("Enter the key");
int key=sc.nextInt();

int result = ob.binarySearch(array, key, 0, n - 1);
if (result == -1)
    System.out.println("Element Not found");
else
    System.out.println("Element found at index " + result);
}
```

**Output:****Case=1**

```
enter array size5
enter the elements of array 15
35
25
95
65
sorted array:[15, 25, 35, 65, 95]
Enter the key
65
Element found at index 3
```

**Case=2**

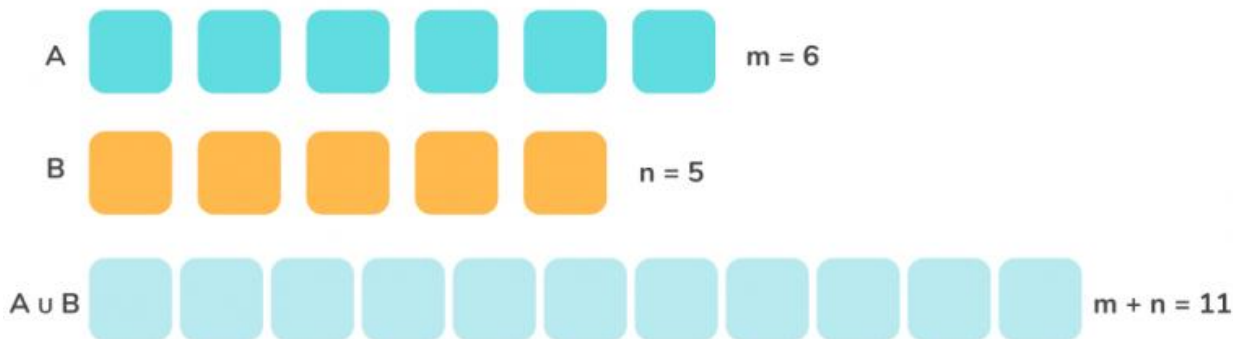
```
enter array size5
enter the elements of array 15
35
25
95
65
sorted array:[15, 25, 35, 65, 95]
Enter the key
30
Element Not found
```

**Applications:**

1. Median of two sorted arrays.
2. Find the fixed point in a given array.
3. Find Smallest Common Element in All Rows.
4. Longest Common Prefix.
5. Koko Eating Bananas.

**1. Median of two sorted arrays:**

- There are two sorted arrays **A** and **B** of sizes **m** and **n** respectively.
- Find the median of the two sorted arrays( The median of the array formed by merging both the arrays).
- **Median:** The middle element is found by ordering all elements in sorted order and picking out the one in the middle (or if there are two middle numbers, taking the mean of those two numbers).

**Examples:**

**Input:** A[] = {1, 4, 5}, B[] = {2, 3}

**Output:** 3

**Explanation:**

Merging both the arrays and arranging in ascending:

[1, 2, 3, 4, 5]

Hence, the median is 3

**Input:** A[] = {1, 2, 3, 4}, B[] = {5, 6}

**Output:** 3.5

**Explanation:**

Union of both arrays:

{1, 2, 3, 4, 5, 6}

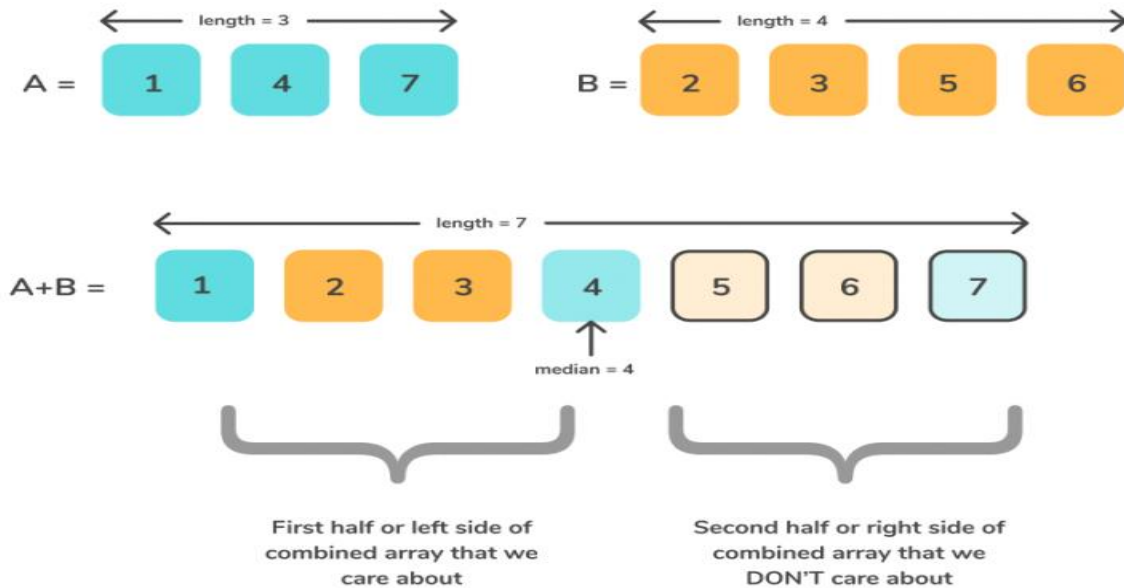
Median =  $(3 + 4) / 2 = 3.5$

The overall Run time complexity should be  $O(\log (m+n))$ .

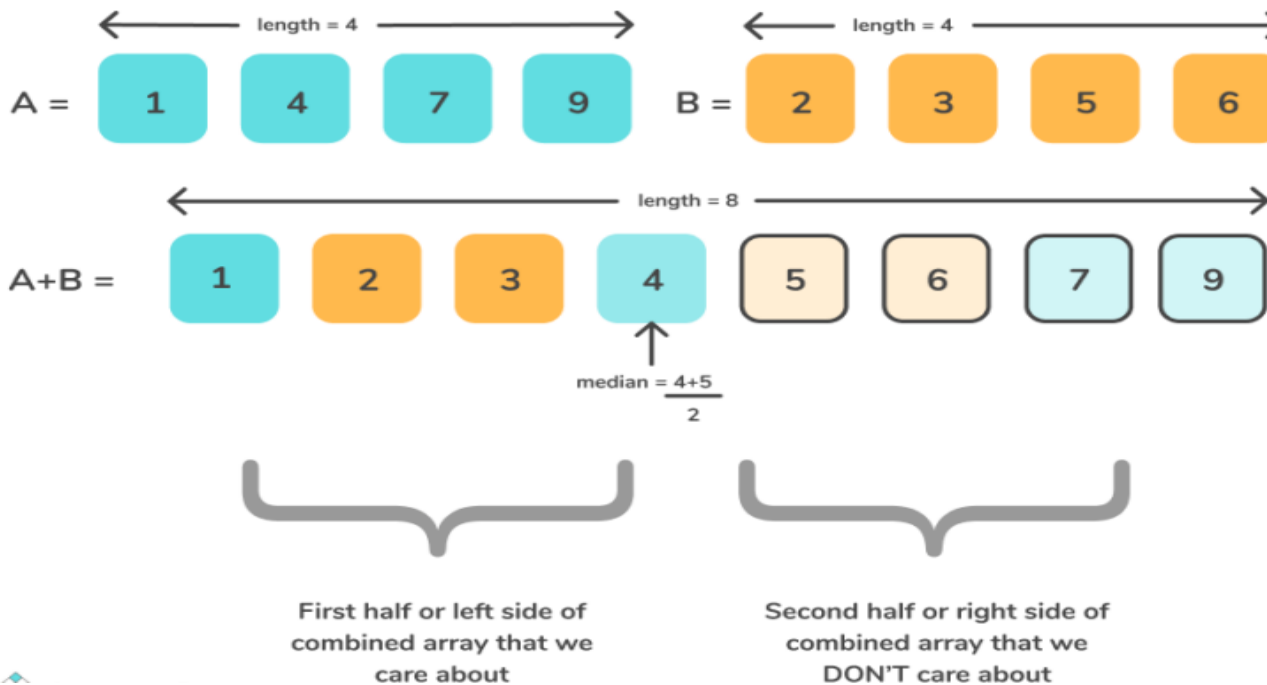
Using Binary search:

The key idea to note here is that both the arrays are **sorted**. Therefore, this leads us to think of binary search. Let us try to understand the algorithm using an example:

$A[] = \{1, 4, 7\}$      $B[] = \{2, 3, 5, 6\}$

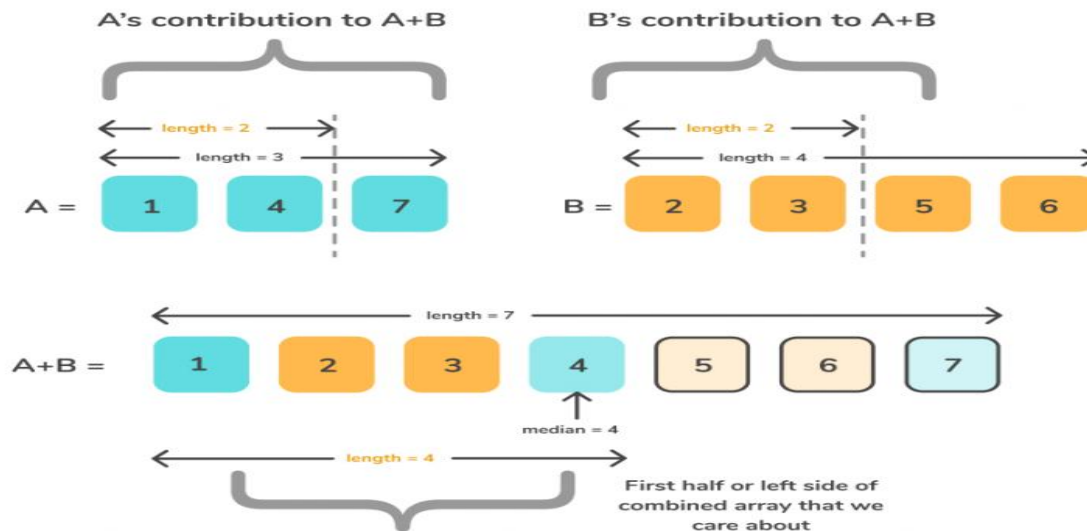


From the above diagram, it can be easily deduced that only the **first half** of the array is needed and the **right half** can be discarded.



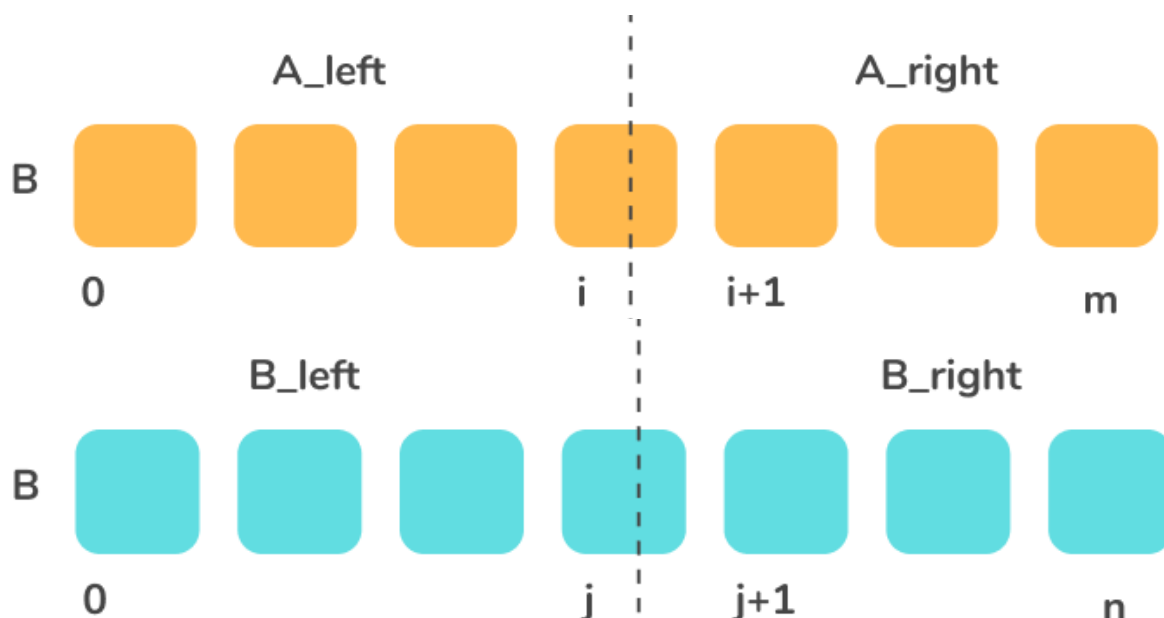
ed.

- Similarly, for an even length merged array, ignore the **right half** and only the **left half** contributes to our final answer.
- Therefore, the motive of our approach is to find which of the elements from both the array helps in contributing to the final answer. Therefore, the **binary search** comes to the rescue, as it can discard a part of the array every time, the elements don't contribute to the median.



### Algorithm

- The first array is of size **n**, hence it can be split into **n + 1** parts.
- The second array is of size **m**, hence it can be split into **m + 1** parts



- As discussed earlier, we just need to find the elements contributing to the left half of the array.
- Since, the arrays are already sorted, it can be deduced that  $A[i - 1] < A[i]$  and  $B[i - 1] < B[i]$ .
- Therefore, we just need to find the index  $i$ , such that  $A[i - 1] \leq B[j]$  and  $B[j - 1] \leq A[i]$ .
- Consider  $mid = (n + m - 1) / 2$  and check if this satisfies the above condition.
- If  $A[i-1] \leq B[j]$  and  $B[j-1] \leq A[i]$  satisfies the condition, return the index  $i$ .
- If  $A[i] < B[j - 1]$ , increase the range towards the right. Hence update  $i = mid + 1$ .
- Similarly, if  $A[i - 1] > B[j]$ , decrease the range towards left. Hence update  $i = mid - 1$ .

|   |   |   |   |   |    |    |    |    |
|---|---|---|---|---|----|----|----|----|
| A | 1 | 3 | 5 | 6 | 7  | 8  | 9  | 11 |
| B | 1 | 4 | 6 | 8 | 12 | 14 | 15 | 17 |

$i$  is the mid of array  $A$  as shown below

|   |   |   |   |          |    |     |    |    |
|---|---|---|---|----------|----|-----|----|----|
| A | 1 | 3 | 5 | $i$<br>6 | 7  | 8   | 9  | 11 |
| B | 1 | 4 | 6 | 8        | 12 | 14  | 15 | 17 |
|   |   |   |   |          |    | $j$ |    |    |

$A[i-1] = 5$   $B[j-1] = 12$ , since  $B[j-1] > A[i]$ , needs to increase

|   |                    |   |   |     |    |          |    |    |
|---|--------------------|---|---|-----|----|----------|----|----|
|   | Discarded elements |   |   |     |    |          |    |    |
| A | 1                  | 3 | 5 | 6   | 7  | $i$<br>8 | 9  | 11 |
| B | 1                  | 4 | 6 | 8   | 12 | 14       | 15 | 17 |
|   |                    |   |   | $j$ |    |          |    |    |

Max of left side will be either  $A[i-1]$  or  $B[j-1]$ , in this case its  $A[i-1] = 7$

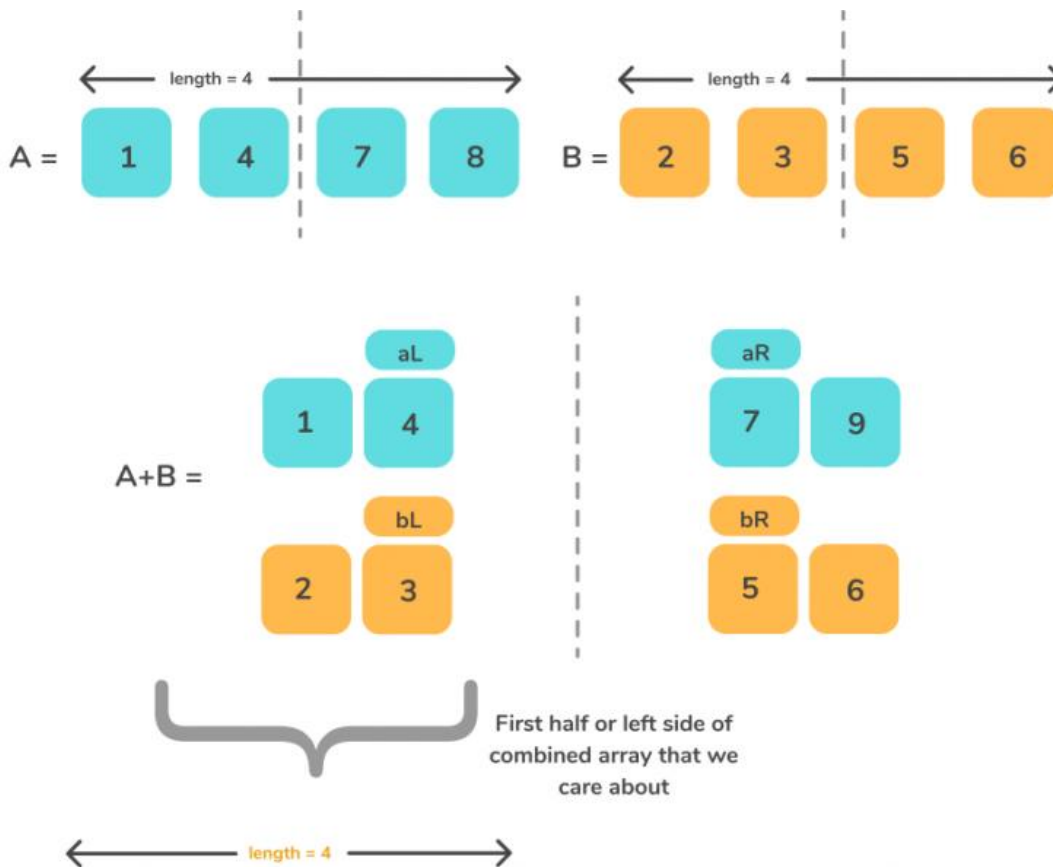
Min of right will be either  $A[i]$  or  $B[j]$ , in this case both are equal which is 8

Since, sum of length is even, we return average of these two which is 7.5

:

**Few corner cases to take care of :**

- If the size of any of the arrays is **0**, return the median of the non-zero sized array.
- If the size of smaller array is **1**,
- If the size of the larger array is also one, simply return the median as the mean of both the elements.
- Else, if size of larger array is odd, adding the element from first array will result in size even, hence median will be affected if and only if, the element of the first array lies between ,  **$M / 2$ th** and  **$M/2 + 1$ th** element of **B[]**.
- Similarly, if the size of the larger array is even, check for the element of the smaller array,  **$M/2$ th element** and  **$M / 2 + 1$  th** element.
- If the size of smaller array is **2**,
- If a larger array has an odd number of elements, the median can be either the **middle** element **or** the median of elements of smaller array and  **$M/2 - 1$ th** element or minimum of the second element of **A[]** and  **$M/2 + 1$  th** array.



**Java program for Median Of Two Sorted Array (Binary search approach)****MedianOfTwoSortedArrays.java**

```
import java.util.*;
public class MedianOfTwoSortedArrays
{
    private static double findMedianSortedArrays(int[] nums1, int[] nums2)
    {
        // Check if num1 is smaller than num2 If not, then we will swap num1 with num2
        if (nums1.length > nums2.length)
        {
            return findMedianSortedArrays(nums2, nums1);
        }
        // Lengths of two arrays
        int m = nums1.length;
        int n = nums2.length;
        // Pointers for binary search
        int start = 0;
        int end = m;
        // Binary search starts from here
        while (start <= end)
        {
            // Partitions of both the array
            int partitionNums1 = (start + end) / 2;
            int partitionNums2 = (m + n + 1) / 2 - partitionNums1;

            // Edge cases there are no elements left on the left side after partition
            int maxLeftNums1 = partitionNums1 == 0 ? Integer.MIN_VALUE : nums1[partitionNums1 - 1];

            // If there are no elements left on the right side after partition
            int minRightNums1 = partitionNums1 == m ? Integer.MAX_VALUE : nums1[partitionNums1];

            // Similarly for nums2
            int maxLeftNums2 = partitionNums2 == 0 ? Integer.MIN_VALUE : nums2[partitionNums2 - 1];

            int minRightNums2 = partitionNums2 == n ? Integer.MAX_VALUE : nums2[partitionNums2];
            // Check if we have found the match

            if (maxLeftNums1 <= minRightNums2 && maxLeftNums2 <= minRightNums1)
            {
                // Check if the combined array is of even/odd length
                if ((m + n) % 2 == 0)
                {

```

```
        return (Math.max(maxLeftNums1, maxLeftNums2)+Math.min(minRightNums1, minRightNums2)) / 2.0;
    }
    else {
        return Math.max(maxLeftNums1, maxLeftNums2);
    }
}
// If we are too far on the right, we need to go to left side
else if (maxLeftNums1 > minRightNums2) {
    end = partitionNums1 - 1;
}
// If we are too far on the left, we need to go to right side
else {
    start = partitionNums1 + 1;
}
}
// If we reach here, it means the arrays are not sorted
throw new IllegalArgumentException();
}
public static void main(String[] args)
{
    Scanner sc=new Scanner(System.in);
    System.out.println("enter the size of the 1st array");
    int n = sc.nextInt();
    int nums1[]=new int[n];
    System.out.println("enter the elements of 1st array ");

    for(int i=0;i<n;i++)
    {
        nums1[i] =sc.nextInt();
    }
    System.out.println("enter the size of the 2nd array");

    int m = sc.nextInt();
    int nums2[]=new int[m];
    System.out.println("enter the elements of 2nd array ");

    for(int i=0;i<m;i++)
    {
        nums2[i] =sc.nextInt();
    }
    System.out.println("The Median of two sorted arrays is :"+ findMedianSortedArrays(nums1, nums2));
}
```

**Example:****Input=**

enter the size of the 1st array

4

enter the elements of 1st array

1 4 7 8

enter the size of the 2nd array

4

enter the elements of 2nd array

2 3 5 6

**Output=**

The Median of two sorted arrays is :4.5

## 2. Find the fixed point in a given array:

- Given an array of  $n$  distinct integers sorted in ascending order, write a function that returns a Fixed Point in the array, if there is any Fixed Point present in array, else returns -1.
- Fixed Point in an array is an index  $i$  such that  $arr[i]$  is equal to  $i$ . Note that integers in array can be negative.

### Example 1:

Input: [-10,-5,0,3,7]

Output: 3

Explanation:

For the given array,  $A[0] = -10$ ,  $A[1] = -5$ ,  $A[2] = 0$ ,  $A[3] = 3$ , thus the output is 3.

### Example 2:

Input: [0,2,5,8,17]

Output: 0

Explanation:

$A[0] = 0$ , thus the output is 0.

### Example 3:

Input: [-10,-5,3,4,7,9]

Output: -1

Explanation:

There is no such  $i$  that  $A[i] = i$ , thus the output is -1.

## Approach

1. Initialize two pointers:  $left = 0$  and  $right = n - 1$ .
2. Find the middle index  $mid = (left + right) / 2$ .
3. Compare  $arr[mid]$  with  $mid$ :
  - If  $arr[mid] == mid$ , then  $mid$  is the fixed point. Return  $mid$ .
  - If  $arr[mid] < mid$ , then the fixed point, if it exists, must be in the **right half**. Search from  $mid + 1$  to  $right$ .
  - If  $arr[mid] > mid$ , then the fixed point, if it exists, must be in the **left half**. Search from  $left$  to  $mid - 1$ .
4. Repeat the above steps until the fixed point is found or the search space becomes empty.
5. If no fixed point exists, return -1.

## Complexity Analysis

- **Time Complexity:**  $O(\log n)$  (Binary Search)
- **Space Complexity:**  $O(1)$  (Iterative approach)

**Java program for Find the Fixed point in an array:****Findfixedpoint.java**

```
import java.util.*;
class Findfixedpoint
{
    static int fixedpoint(int arr[], int low, int high)
    {
        if (high >= low) {
            int mid = low + (high - low) / 2;
            if (mid == arr[mid])
                return mid;
            int res = -1;
            if (mid + 1 <= arr[high])
                res = fixedpoint(arr, (mid + 1), high);
            if (res != -1)
                return res;
            if (mid - 1 >= arr[low])
                return fixedpoint(arr, low, (mid - 1));
        }

        /* Return -1 if there is no Fixed Point */
        return -1;
    }

    // main function
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        System.out.println("enter array size");
        int n = sc.nextInt();
        int array[]=new int[n];

        System.out.println("enter the elements of array ");

        for(int i=0;i<n;i++)
        {
            array[i] =sc.nextInt();
        }
    }
}
```

```
Arrays.sort(array);

// Printing the array after sorting
System.out.println("sorted array["+ Arrays.toString(array));
System.out.println("Fixed Point is " + fixedpoint(array, 0, n - 1));
}
}
```

**Example-1:****Input=**

enter array size

10

enter the elements of array

11 30 50 0 3 100 -10 -1 10 102

sorted array:[-10, -1, 0, 3, 10, 11, 30, 50, 100, 102]

**Output=**

Fixed Point is 3.

**Example-2:****Input=**

enter array size

6

enter the elements of array

3 9 4 7 -5 -10

sorted array:[-10, -5, 3, 4, 7, 9]

**Output=**

Fixed Point is -1

**Example-3:****Input=**

enter array size

5

enter the elements of array

8 2 5 17 0

sorted array:[0, 2, 5, 8, 17]

**Output=**

Fixed Point is 0

### 3. Find Smallest Common Element in All Rows.

Given a matrix where every row is sorted in increasing order. Write a function that finds and returns a smallest common element in all rows. If there is no common element, then returns -1.

#### Example-1:

**Input:** mat [4][5] = { { 1, 2, 3, 4, 5},  
                          { 2, 4, 5, 8, 10},  
                          { 3, 5, 7, 9, 11},  
                          { 1, 3, 5, 7, 9}  
                          };

**Output:** 5

#### Approach

1. Traverse each element of the **first row** from left to right.
2. Treat the current element as a candidate for the smallest common element.
3. For each of the remaining rows, use **Binary Search** to check whether the candidate exists.
4. If the candidate is found in **every row**, return it immediately since the first row is sorted and this is the smallest possible common element.
5. If the candidate is not found in any one of the rows, continue with the next element of the first row.
6. If all elements of the first row are checked and none is common to every row, return -1.

#### Algorithm

Algorithm: Find Smallest Common Element

Input: A sorted matrix of R rows and C columns.

Output: Smallest common element or -1.

Step 1: For each element x in the first row  
        found = true

Step 2: For every remaining row  
        Perform Binary Search for x.

        If x is not found  
            found = false  
            Break

Step 3: If found is true  
        Return x

Step 4: After checking all elements of the first row,  
        Return -1.

**Java Program for Find the Smallest Common Element Using Binary Search:SmallestCommonElement.java**

```
import java.util.*;
class SmallestCommonElement
{
    private boolean binarySearch(int[] arr, int low, int high, int target)
    {
        while(low <= high) {
            int mid = (low + high)/2;
            if(arr[mid] == target)
            {
                return true;
            }
            else if(arr[mid] < target)
            {
                low = mid+1;
            } else {
                high = mid-1;
            }
        }
        return false;
    }
    public int smallestCommonElement(int[][] mat)
    {
        if(mat.length == 1) return mat[0][0];
        for(int a : mat[0]) {
            int count = 0;
            for(int i=1; i<mat.length; i++) {
                if(binarySearch(mat[i], 0, mat[i].length-1, a))
                {
                    count++;
                } else {
                    break;
                }
            }
            if(count == mat.length-1) return a;
        }
        return -1;
    }
    public static void main(String[] args)
```

```

{
    Scanner sc=new Scanner(System.in);
    System.out.println( "enter size of two dimensional matrix" );
    System.out.println( "enter row size of matrix " );
    int m=sc.nextInt();
    System.out.println( "enter column size of matrix " );
    int n=sc.nextInt();
    int[][] arr = new int[m][n];
    System.out.println( "enter the elements " );
    for(int i=0;i<m;i++)
        for(int j=0;j<n;j++)
            arr[i][j] = sc.nextInt();
    System.out.println( "smallest common element :"+new SmallestCommonElement().smallestCommonElement(arr) );
}
}

```

**Exmaple-1:****input=**

enter size of two dimensional matrix

enter row size of matrix

4

enter column size of matrix

5

enter elements

1 2 3 4 5

2 4 5 8 10

3 5 7 9 11

1 3 5 7 9

**output=**

smallest common element :5

**Exmaple-2:****input=**

enter size of two dimensional matrix

enter row size of matrix

2

enter column size of matrix

2

enter elements for matrix

1 2

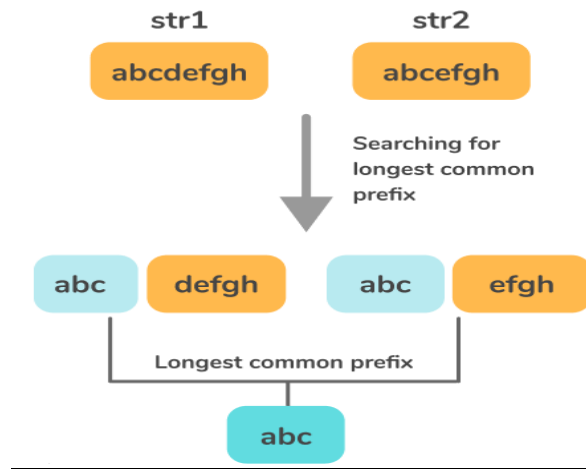
3 4

**output=**

smallest common element :-1

#### 4. Longest Common Prefix

- Given the array of strings S[], you need to find the longest string S which is the prefix of ALL the strings in the array.
- **Longest common prefix (LCP)** for a pair of strings S1 and S2 is the longest string S which is the prefix of both S1 and S2.
- For Example: longest common prefix of “**abcdefgh**” and “**abcefg**h” is “**abc**”.

**Example-1:**

**Input:** S[] = {"abcdefgh", "abcefg"}

**Output:** "abc"

**Explanation:** Explained in the image description above

**Example-2:**

**Input:** S[] = {"abcdefgh", "aefghijk", "abcefg"}

**Output:** "a"

**Approach**

The problem is solved using the **Divide and Conquer** technique.

1. If the array contains only one string, return that string as the longest common prefix.
2. Divide the array of strings into two equal halves.
3. Recursively find the longest common prefix of the left half.
4. Recursively find the longest common prefix of the right half.
5. Compare the two prefixes character by character to obtain their common prefix.
6. Return the common prefix obtained after merging the two halves.
7. Continue until the longest common prefix of all strings is found.

## Binary Search Approach

### Algorithm:

- Consider the string with the smallest length. Let the length be **L**.
- Consider a variable **low** = 0 and **high** = **L** - 1.
- Perform binary search:
- Divide the string into two halves, i.e. **low** - **mid** and **mid** + 1 to **high**.
- Compare the substring upto the **mid** of this smallest string to every other character of the remaining strings at that index.
- If the substring from 0 to **mid** - 1 is common among all the substrings, update **low** with **mid** + 1, else update **high** with **mid** - 1
- If **low** == **high**, terminate the algorithm and return the substring from 0 to **mid**.

### Time Complexity:

Let: **N** = Number of strings **M** = Length of the shortest string

The recurrence relation is:  $T(N) = 2T(N/2) + O(M)$

Using the Master Theorem, **Time Complexity** =  $O(N \times M)$

because there are **log N** levels of recursion, and the total work done in comparing prefixes across each level is proportional to the total number of strings times the prefix length.

where  $O(M)$  is the time taken by `commonPrefix()` to compare two prefixes.

Applying the Master Theorem:

- $a = 2$
- $b = 2$
- $f(N) = O(M)$  (independent of  $N$ )

The recursion tree has **log N** levels.

At each level:

- There are more calls, but the total number of `commonPrefix()` operations across that level is proportional to **N/2**.
- Each `commonPrefix()` can compare up to **M** characters.
- Total work per level:  $O(N \times M)$
- There are  $\log N$  levels, but each level performs proportionally fewer merges at larger prefix sizes.

Summing the work gives:

$$\begin{aligned} & (N/2 \times M) + (N/4 \times M) + (N/8 \times M) + \dots \\ &= NM (1/2 + 1/4 + 1/8 + \dots) \\ &= NM \end{aligned}$$

Since:  $1/2 + 1/4 + 1/8 + \dots = 1$

the total becomes  $O(N \times M)$

Therefore, **Time Complexity:  $O(N \times M)$**

**Space Complexity:  $O(\log N)$**  (due to the recursion stack)

### **Java program for LongestCommonPrefix using Binary search approach**

#### **LongestCommonPrefix.java**

```
import java.util.*;

class LongestCommonPrefix
{
    public String longestCommonPrefix(String[] strs)
    {
        if (strs == null || strs.length == 0)
            return "";
        return longestCommonPrefix(strs, 0, strs.length - 1);
    }

    private String longestCommonPrefix(String[] strs, int l, int r)
    {
        if (l == r)
        {
            return strs[l];
        }
        else
        {
            int mid = (l + r)/2;
            String lcpLeft = longestCommonPrefix(strs, l, mid);
            String lcpRight = longestCommonPrefix(strs, mid + 1, r);
            return commonPrefix(lcpLeft, lcpRight);
        }
    }

    String commonPrefix(String left, String right)
    {
        int min = Math.min(left.length(), right.length());
        for (int i = 0; i < min; i++)
        {
            if (left.charAt(i) != right.charAt(i))
                return left.substring(0, i);
        }
        return left;
    }
}
```

```
        return left.substring(0, i);
    }
    return left.substring(0, min);
}

public static void main(String args[])
{
    Scanner sc= new Scanner(System.in);
    System.out.println("Enter Strings");
    String[] words = sc.nextLine().split(" ");

    System.out.println("Longest common Prefix is: "+new
LongestCommonPrefix().longestCommonPrefix(words));
}
}
```

**Example-1:****input:**

Enter Strings  
fly flower flow

**output:**

Longest common Prefix is: fl

**Example-2:****input:**

Enter Strings  
f c i

**output:**

Longest common Prefix is:

## 5. Koko Eating Bananas:

- Koko loves to eat bananas. There are  $n$  piles of bananas, the  $i^{\text{th}}$  pile has **piles[i]** bananas. The guards have gone and will come back in  $h$  hours.
- Koko can decide her bananas-per-hour eating speed of **k**. Each hour, she chooses some pile of bananas and eats **k** bananas from that pile. If the pile has less than **k** bananas, she eats all of them instead and will not eat any more bananas during this hour.
- Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.
- Return *the minimum integer k such that she can eat all the bananas within h hours.*

### Example 1:

**Input:** piles = [3,6,7,11], h = 8

**Output:** 4

### Example 2:

**Input:** piles = [30,11,23,4,20], h = 5

**Output:** 30

### Example 3:

**Input:** piles = [30,11,23,4,20], h = 6

**Output:** 23

### Input:

piles = [30,11,23,4,20], H = 6

### output:

23



Koko will eat bananas in this way to eat all bananas in 6 hours:

**First hour:** 23

**Second hour:** 7

**Third hour:** 11

**Fourth hour:** 23

### Approach for Koko Eating Bananas

The problem is solved using the **Binary Search** technique on the possible eating speed.

1. The minimum possible eating speed is **1 banana per hour**.
2. The maximum possible eating speed is equal to the **largest pile of bananas**.
3. Apply Binary Search on the range [**1, maximum pile**].
4. For each middle value ( $mid$ ), assume it is Koko's eating speed.
5. Calculate the total number of hours required to eat all the banana piles at this speed.
6. If the total hours are **less than or equal to  $H$** , then this speed is sufficient. Try to find a smaller valid speed by searching the left half ( $high = mid - 1$ ).
7. Otherwise, the speed is too slow. Search the right half ( $low = mid + 1$ ).
8. Repeat the process until  $low > high$ .
9. The value of  $low$  is the minimum eating speed required to finish all bananas within  $H$  hours.

#### **Algorithm: Koko Eating Bananas**

Input: Banana piles[] and H hours

Output: Minimum eating speed

Step 1: Set  $low = 1$ .

Step 2: Set  $high = \text{Maximum element in the piles array}$ .

Step 3: While  $low \leq high$

$mid = (low + high) / 2$

Calculate total hours required to eat all piles  
at speed =  $mid$ .

If  $totalHours \leq H$

$high = mid - 1$

Else

$low = mid + 1$

Step 4: Return  $low$ .

### Complexity Analysis

Let:

- $N$  = Number of banana piles
- $M$  = Maximum number of bananas in any pile
- Finding the maximum pile:  $O(N)$
- Binary Search iterations:  $O(\log M)$
- Each iteration calculates total hours in  $O(N)$

Therefore, **Time Complexity** =  $O(N \log M)$

**Space complexity:** The space complexity of the above code is **O(1)** because we are using only a variable to store the answer.

**Java program for Kokoeatingbananas:**      **Kokoeatingbanana.java**

```
import java.util.*;
public class Kokoeatingbanana
{
    public static int calculateTotalHours(int[] v, int hourly)
    {
        int totalH = 0;
        int n = v.length;
        //find total hours:
        for (int i = 0; i < n; i++) {
            totalH += Math.ceil((double)(v[i]) / (double)(hourly));
        }
        return totalH;
    }

    public static int minEatingSpeed(int[] piles, int H)
    {
        int low = 1, high = Arrays.stream(piles).max().getAsInt();
        //apply binary search:
        while (low <= high)
        {
            int mid = (low + high) / 2;
            int totalH = calculateTotalHours(piles, mid);
            if (totalH <= H)
            {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        return low;
    }

    public static void main(String[] args) throws Exception
    {
        Scanner sc=new Scanner(System.in);
        int n = sc.nextInt();
        int arr[]=new int[n];

        for(int i=0;i<n;i++)
```

```

    {
        arr[i] = sc.nextInt();
    }
    int H = sc.nextInt();
    int ans = minEatingSpeed(arr, H);
    System.out.println(ans);
}
}

```

input=

4

3 6 7 11

8

output=

4

/\* Tracing

Input: Piles = [3, 6, 2, 8]

H = 7

-----

**Step 1:**

Find Search Space

low = 1

high = max(Piles) = 8

-----

**Iteration 1**

low = 1

high = 8

mid = (1 + 8) / 2 = 4

Speed = 4 bananas/hour

| Pile | Hours Required |
|------|----------------|
| 3    | ceil(3/4) = 1  |
| 6    | ceil(6/4) = 2  |
| 2    | ceil(2/4) = 1  |
| 8    | ceil(8/4) = 2  |

Total Hours = 1 + 2 + 1 + 2 = 6

6 <= 7

Possible

high = mid - 1 = 3

-----

**Iteration 2**

```
low = 1
high = 3
```

```
mid = (1 + 3) / 2 = 2
```

```
Speed = 2 bananas/hour
```

| Pile | Hours Required         |
|------|------------------------|
| 3    | $\text{ceil}(3/2) = 2$ |
| 6    | $\text{ceil}(6/2) = 3$ |
| 2    | $\text{ceil}(2/2) = 1$ |
| 8    | $\text{ceil}(8/2) = 4$ |

```
Total Hours = 2 + 3 + 1 + 4 = 10
10 > 7
```

```
Not Possible
```

```
low = mid + 1 = 3
```

### Iteration 3

```
low = 3
high = 3
mid = (3 + 3) / 2 = 3
Speed = 3 bananas/hour
```

| Pile | Hours Required         |
|------|------------------------|
| 3    | $\text{ceil}(3/3) = 1$ |
| 6    | $\text{ceil}(6/3) = 2$ |
| 2    | $\text{ceil}(2/3) = 1$ |
| 8    | $\text{ceil}(8/3) = 3$ |

```
Total Hours = 1 + 2 + 1 + 3 = 7
```

```
7 <= 7
```

```
Possible
```

```
high = mid - 1 = 2
```

```
Loop Ends
```

```
low = 3
high = 2
```

```
Since low > high, stop.
```

```
Answer = low = 3
```

```
Minimum Eating Speed = 3 bananas/hour
```

**II. Greedy Method:**

- Among all the algorithmic approaches, the simplest and straightforward approach is the Greedy method.
- In this approach, the decision is taken on the basis of current available information without worrying about the effect of the current decision in future.
- Greedy algorithms build a solution part by part, choosing the next part in such a way, that it gives an immediate benefit.
- This approach never reconsiders the choices taken previously
- This approach is mainly used to solve optimization problems.
- Greedy method is easy to implement and quite efficient in most of the cases.
- Greedy algorithm is an algorithmic paradigm based on heuristic that follows local optimal choice at each step with the hope of finding global optimal solution.
- In many problems, it does not produce an optimal solution though it gives an approximate (near optimal) solution in a reasonable time.

**Control abstraction (pseudo code) for Greedy Method:**

```
1  Algorithm Greedy(a, n)
2  // a[1 : n] contains the n inputs.
3  {
4      solution := ∅; // Initialize the solution.
5      for i := 1 to n do
6          {
7              x := Select(a);
8              if Feasible(solution, x) then
9                  solution := Union(solution, x);
10         }
11     return solution;
12 }
```

**Three important activities:**

1. A selection of solution from the given input domain is performed, i.e.  $s := \text{select}(a)$ .
2. The feasibility of the solution is performed, by using feasible '(solution, s)' and then all feasible solutions are obtained.
3. From the set of feasible solutions, the particular solution that minimizes or maximizes the given objection function is obtained. Such a solution is called optimal solution

**Applications:**

1. Minimum product subset of an array
2. Best Time to Buy and Sell Stock
3. 0/1 Knapsack Problem
4. Minimum cost spanning trees
5. Single source shortest path Problem

**1. Minimum product subset of an array**

Given an array  $a$ , we have to find the minimum product possible with the subset of elements present in the array. The minimum product can be a single element also.

**Examples:**

**Input :**  $a[] = \{ -1, -1, -2, 4, 3 \}$

**Output :** -24

Explanation : Minimum product will be  $(-2 * -1 * -1 * 4 * 3) = -24$

**Input :**  $a[] = \{ -1, 0 \}$

**Output :** -1

Explanation : -1(single element) is minimum product possible

**Input :**  $a[] = \{ 0, 0, 0 \}$

**Output :** 0

A simple solution is to generate all subsets, find the product of every subset and return the minimum product.

A better solution is to use the below facts.

- >If there are even number of negative numbers and no zeros, the result is the product of all except the largest valued negative number.
- >If there are an odd number of negative numbers and no zeros, the result is simply the product of all.
- >If there are zeros and positive, no negative, the result is 0. The exceptional case is when there is no negative number and all other elements positive then our result should be the first minimum positive number.

**Complexity Analysis:**

**Time Complexity:**  $O(n)$

**Auxiliary Space:**  $O(1)$

**Java program to find Minimum product of a subset**      **MinProductSubset.java**

```
import java.util.*;

class MinProductSubset
{
    static int minProductSubset(int a[], int n)
    {
        if (n == 1)
            return a[0];

        /*Find count of negative numbers,count of zeros, maximum valued negative number, minimum valued
        positive number and product of non-zero numbers */

        int negmax = Integer.MIN_VALUE;
        int posmin = Integer.MAX_VALUE;
        int count_neg = 0;
        int count_zero = 0;
        int product = 1;

        for (int i = 0; i < n; i++)
        {

            // if number is zero,count it but dont multiply

            if (a[i] == 0) {
                count_zero++;
                continue;
            }

            // count the negative numbers and find the max negative number
            if (a[i] < 0) {
                count_neg++;
                negmax = Math.max(negmax, a[i]);
            }

            // find the minimum positive number
            if (a[i] > 0 && a[i] < posmin)
                posmin = a[i];

            product *= a[i];
        }

        // if there are all zeroes or zero is present but no negative number is present
```

```

if (count_zero == n || (count_neg == 0 && count_zero > 0))
    return 0;

```

```

// If there are all positive

```

```

if (count_neg == 0)
    return posmin;

```

```

// If there are even number except

```

```

// zero of negative numbers

```

```

if (count_neg % 2 == 0 && count_neg != 0) {

```

```

    // Otherwise result is product of

```

```

    // all non-zeros divided by maximum

```

```

    // valued negative.

```

```

    product = product / negmax;

```

```

}

```

```

return product;

```

```

}

```

```

public static void main(String[] args)

```

```

{

```

```

    Scanner sc=new Scanner(System.in);

```

```

    System.out.println("enter size of the array");

```

```

    int n=sc.nextInt();

```

```

    System.out.println("enter elements");

```

```

    int a[]=new int[n];

```

```

    for(int i=0;i<n;i++)

```

```

    {

```

```

        a[i]=sc.nextInt();

```

```

    }

```

```

    System.out.println(minProductSubset(a, n));

```

```

}

```

```

}

```

**input=**

enter size of the array

5

enter elements

-1 -1 -2 4 3

**output=**

-24

## 2. Best Time to Buy and Sell Stock

### **Type I: At most one transaction is allowed**

Given an array `prices[]` of length `N`, representing the prices of the stocks on different days, the task is to find the maximum profit possible for buying and selling the stocks on different days using transactions where at most one transaction is allowed.

Note: Stock must be bought before being sold.

Input: `prices[] = {7, 1, 5, 3, 6, 4}`

Output: 5

Explanation:

The lowest price of the stock is on the 2nd day, i.e. price = 1. Starting from the 2nd day, the highest price of the stock is witnessed on the 5th day, i.e. price = 6.

Therefore, maximum possible profit =  $6 - 1 = 5$ .

Input: `prices[] = {7, 6, 4, 3, 1}`

Output: 0

Explanation: Since the array is in decreasing order, no possible way exists to solve the problem.

**Time Complexity:  $O(N)$ . Where  $N$  is the size of prices array.**

**Auxiliary Space:  $O(1)$ . We do not use any extra space.**

### **Java program for Best Time to Buy and Sell Stock with at most one transaction**

**BTBS\_Atmostonetime.java**

```
import java.util.*;
```

```
class BTBS_Atmostonetime
{
    static int maxProfit(int prices[], int n)
    {
        int minprice=Integer.MAX_VALUE;
        int max_profit = 0;

        for (int i = 0; i < n; i++)
        {
            if (prices[i]< minprice)
                minprice= prices[i];

            else if (prices[i] -minprice > max_profit)
```

```
    max_profit = prices[i] - minprice;
}
return max_profit;
}
public static void main(String args[])
{
    Scanner sc =new Scanner(System.in);
    System.out.println("Enter array size");

    int size=sc.nextInt();

    int prices[]=new int[size];

    System.out.println("Enter elements");

    for(int i=0;i<size;i++)
    {
        prices[i]=sc.nextInt();
    }
    int max_profit = maxProfit(prices,size);
    System.out.println("Maximum Profit is:" + max_profit);
}
}
```

Input=

Enter array size

6

Enter elements

7 1 5 3 6 4

Output=

Maximum Profit is:5

Input=

Enter array size

6

Enter elements

6 5 4 3 2 1

output

Maximum Profit is:0

**Type II: Infinite transactions are allowed**

Given an array `price[]` of length `N`, representing the prices of the stocks on different days, the task is to find the maximum profit possible for buying and selling the stocks on different days using transactions where any number of transactions are allowed.

**input:** `prices[] = {7, 1, 5, 3, 6, 4}`

**Output:** 7

**Explanation:**

Purchase on 2nd day. Price = 1.

Sell on 3rd day. Price = 5.

Therefore, profit =  $5 - 1 = 4$ .

Purchase on 4th day. Price = 3.

Sell on 5th day. Price = 6.

Therefore, profit =  $4 + (6 - 3) = 7$ .

**Input:** `prices = {1, 2, 3, 4, 5}`

**Output:** 4

**Explanation:**

Purchase on 1st day. Price = 1.

Sell on 5th day. Price = 5.

Therefore, profit =  $5 - 1 = 4$ .

**Approach:** The idea is to maintain a boolean value that denotes if there is any current purchase ongoing or not. If yes, then at the current state, the stock can be sold to maximize profit or move to the next price without selling the stock. Otherwise, if no transaction is happening, the current stock can be bought or move to the next price without buying.

**Java program for Best Time to Buy and Sell Stock with Infinite transactions :****BTBS\_InfiniteTransactions.java**

```
import java.util.*;
class BTBS_InfiniteTransactions
{
    public static void main(String [] args)
    {
        Scanner sc = new Scanner(System.in);
        int n=sc.nextInt();
        int arr[] = new int[n];
        for(int i=0;i<n;i++){
            arr[i]=sc.nextInt();
        }
        System.out.println(prof(arr,n));
    }
    public static int prof(int arr[] , int n){
        int profit=0;
        for(int i=0;i<n-1;i++)
        {
            if(arr[i]<arr[i+1])
            {
                profit+=(arr[i+1]-arr[i]);
            }
        }
        return profit;
    }
}
```

**input=**

7

7 1 5 3 6 4

**Output=**

7

**3. Greedy Knapsack Problem (Fractional Knapsack) :**

- In this problem we have a Knapsack that has a weight limit  $M$ .
- There are items  $i_1, i_2, \dots$ , in each having weight  $w_1, w_2, \dots, w_n$  and some benefit (value or profit) associated with it  $p_1, p_2, \dots, p_n$
- Our objective is to maximize the benefit such that the total weight inside the knapsack is at most  $M$ , and we are also allowed to take an item in fractional part.
- There are  **$n$  items in the store**
  - weight of  **$i$ th item  $w_i > 0$**
  - Profit for  **$i$ th item  $p_i > 0$  and**
  - Capacity of the Knapsack is  **$M$** .
- In this version of Knapsack problem, items can be broken into smaller pieces. So, the thief may take only a fraction  **$x_i$  of  $i$ th item.  $0 \leq x_i \leq 1$**
- The  $i$ th item contributes the weight  **$x_i \cdot w_i$**  to the total weight in the knapsack and profit  **$x_i \cdot p_i$**  to the total profit.
- Hence, the objective of this algorithm is to ***maximize*  $\sum_{i=1}^n (x_i \cdot p_i)$**
- subject to constraint,  **$\sum_{i=1}^n (x_i \cdot w_i) \leq M$**
- It is clear that an optimal solution must fill the knapsack exactly, otherwise we could add a fraction of one of the remaining items and increase the overall profit.
- Thus, an optimal solution can be obtained by  **$\sum_{i=1}^n (x_i \cdot w_i) = M$**
- In this context, first we need to sort those items according to the value of  **$p_i/w_i$ , so that  $(p_{i+1}/w_{i+1}) \leq p_i/w_i$ .**
- **Here,  $x$  is an array to store the fraction of items.**

**Pseudo Code of Greedy Knapsack:**

```

void GreedyKnapsack(float m, int n)
// p[1:n] and w[1:n] contain the profits and weights
// respectively of the n objects ordered such that
// p[i]/w[i] >= p[i+1]/w[i+1]. m is the knapsack
// size and x[1:n] is the solution vector.
{
    for (int i=1; i<=n; i++) x[i] = 0.0; // Initialize x.
    float U = m;
    for (i=1; i<=n; i++) {
        if (w[i] > U) break;
        x[i] = 1.0;
        U -= w[i];
    }
    if (i <= n) x[i] = U/w[i];
}

```

**Analysis:**

If the provided items are already sorted into a decreasing order of  $pi/wi$ , then the while loop takes a time in  $(n)$ ; Therefore, the total time including the sort is in

$O(n \log n)$ .

**Example Problems:**

**Problem -1:** We have 3 items with (weight, profit), Item 1: (10, 60) Item 2: (20, 100) Item 3: (30, 120) Knapsack capacity  $W = 50$ , Goal is Select items (or fractions) to maximize total value.

**Step 1:** Compute profit/weight ratios:

Item 1:  $60 / 10 = 6.0$

Item 2:  $100 / 20 = 5.0$

Item 3:  $120 / 30 = 4.0$

Sort by ratio (descending): Item1, Item2, Item3.

**Step 2:** Greedy selection:

1) Take Item 1 fully: weight used = 10, profit = 60. Remaining capacity = 40.

2) Take Item 2 fully: weight used = 20, profit = 100. Remaining capacity = 20.

3) Take Item 3 partially: only 20 out of 30 weight can be taken. Fraction =  $20/30 = 2/3$ .

Value from fraction =  $120 * (20/30) = 80$ .

**Final selection and total value:**

**Items chosen:** Item1 (full), Item2 (full), Item3 (2/3). → **Feasible Solution:**  $(x_1, x_2, x_3) = (1, 1, 2/3)$

**Total profit ( $\sum pi*xi$ )** =  $1*60 + 1*100 + 2/3 *120 = 240$ .

**Answer:** Maximum Profit achievable = 240.

**Problem -2:** We have 4 items with (weight, profit): Item 1: (12, 24), Item 2: (7, 13), Item 3: (11, 23), Item 4: (8, 15) and Knapsack capacity  $W = 26$

**Step 1:** Compute profit /weight ratios:

Item 1:  $24 / 12 = 2.0000$

Item 2:  $13 / 7 \approx 1.8571$

Item 3:  $23 / 11 \approx 2.0909$

Item 4:  $15 / 8 = 1.8750$

Sort by ratio (descending): Item3, Item1, Item4, Item2.

**Step 2: Greedy selection:**

- 1) Take Item 3 fully: weight used = 11, profit = 23. Remaining capacity = 15.
- 2) Take Item 1 fully: weight used = 12, profit = 24. Remaining capacity = 3.
- 3) Item 4: weight = 8, but only 3 capacity remains. Take fraction 3/8.  
Value from fraction =  $15 * (3/8) = 5.625$ .
- 4) Item 2 is not taken (no capacity left).

**Final selection and total value:**

Items chosen: Item3 (full), Item1 (full), Item4 (3/8). → **Feasible Solution:**  $(x_1, x_2, x_3, x_4) = (1, 0, 1, 3/8)$

**Total profit ( $\sum p_i x_i$ )** =  $1*24 + 0*13 + 1*23 + 3/8*15 = 52.625$ .

**Answer:** Maximum profit achievable  $\approx 52.625$ .

**Java Program for Fractional knapsack : Greedy\_Knapsack.java**

```
import java.util.Scanner;

class Greedy_Knapsack {

    // Function to implement Fractional Knapsack
    static void greedyKnapsack(float profit[], float weight[], int n, float capacity) {

        float[] x = new float[n];

        // Initialize solution vector
        for (int i = 0; i < n; i++)
            x[i] = 0;

        float U = capacity;
        int i;

        // Select complete objects
        for (i = 0; i < n; i++) {

            if (weight[i] > U)
                break;

            x[i] = 1.0f;
            U = U - weight[i];
        }

        // Select fractional object
```

```
        if (i < n)
            x[i] = U / weight[i];

        // Calculate Maximum Profit
        float totalProfit = 0;

        for (i = 0; i < n; i++)
            totalProfit += x[i] * profit[i];

        // Display Feasible Solution
        System.out.println("\nFeasible Solution");

        for (i = 0; i < n; i++) {
            System.out.printf("x%d = %.2f\n", (i + 1), x[i]);
        }

        // Display Maximum Profit
        System.out.printf("\nMaximum Profit = %.2f\n", totalProfit);
    }

    public static void main(String args[]) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of objects: ");
        int n = sc.nextInt();

        float[] profit = new float[n];
        float[] weight = new float[n];

        System.out.println("Enter Profit and Weight of each object");
        System.out.println("(Objects must be entered in decreasing order of Profit/Weight Ratio)");

        for (int i = 0; i < n; i++) {
            System.out.print("Profit of Object " + (i + 1) + ": ");
            profit[i] = sc.nextFloat();

            System.out.print("Weight of Object " + (i + 1) + ": ");
            weight[i] = sc.nextFloat();
        }

        System.out.print("Enter Knapsack Capacity: ");
        float capacity = sc.nextFloat();
```

```
        greedyKnapsack(profit, weight, n, capacity);  
        sc.close();  
    }  
}
```

**Example:**

Input=

Enter number of objects:

3

Enter Profit and Weight of each object

(Objects must be entered in decreasing order of Profit/Weight Ratio)

Profit of Object 1: 60

Weight of Object 1: 10

Profit of Object 2: 100

Weight of Object 2: 20

Profit of Object 3: 120

Weight of Object 3: 30

Enter Knapsack Capacity:

50

output=

Feasible Solution

 $x_1 = 1.00$  $x_2 = 1.00$  $x_3 = 0.67$ 

Maximum Profit = 240.00

**Graphs:**

A graph can be represented as a collection of vertices with edges joining them. The most popular types of graphs:

- **Undirected Graph:** An undirected graph is one where the edges do not indicate in the same direction, making it bidirectional instead of unidirectional. It's also feasible to consider it as a graph with V vertices and E edges, each uniting two separate vertices.
- **Connected Graph:** A connected graph is one when there is invariably a path from one vertex to another. Also, we can say that a graph is connected if we can go to any vertex from any different vertex by pursuing edges in either direction.
- **Directed Graph:** A graph is considered a directed graph if all the edges present between any nodes or vertices have a defined direction.

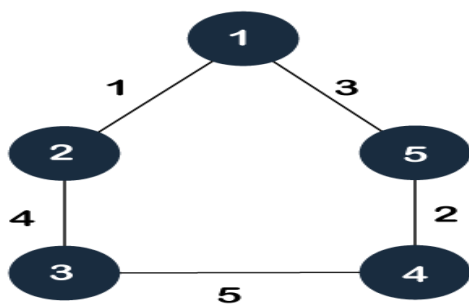
**Spanning Tree:**

- A spanning tree is known as a subgraph of an undirected connected graph that possesses all of the graph's edges or vertices with the rarest feasible edges. If a vertex is missing, then it is not a spanning tree.
- A complete undirected graph possesses  $n^{(n-2)}$  number of spanning trees, so if we have  $n = 4$ , the highest number of potential spanning trees is equivalent to  $4^{4-2} = 16$ . Thus, 16 spanning trees can be constructed from a complete graph with 4 vertices.

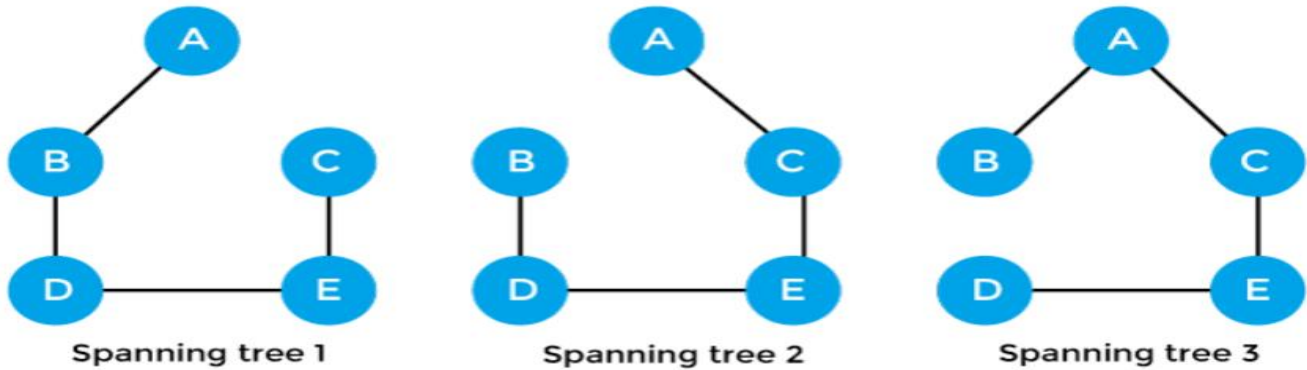
**Example of Spanning Tree:**

Now, let's walk through the below example and try to understand the spanning tree and how it works:

Let the real graph be:



The following are some examples of spanning trees that can be generated from the graph above:



### Properties of Spanning Tree

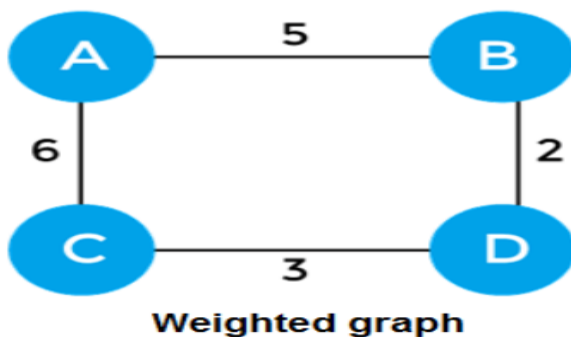
- There can be more than one spanning tree of a connected graph G.
- A spanning tree does not have any cycles or loop.
- A spanning tree is **minimally connected**, so removing one edge from the tree will make the graph disconnected.
- A spanning tree is **maximally acyclic**, so adding one edge to the tree will create a loop.
- There can be a maximum  $n^{n-2}$  number of spanning trees that can be created from a complete graph.
- A spanning tree has  $n-1$  edges, where 'n' is the number of nodes.
- If the graph is a complete graph, then the spanning tree can be constructed by removing maximum  $(e-n+1)$  edges, where 'e' is the number of edges and 'n' is the number of vertices.

### Minimum Spanning tree

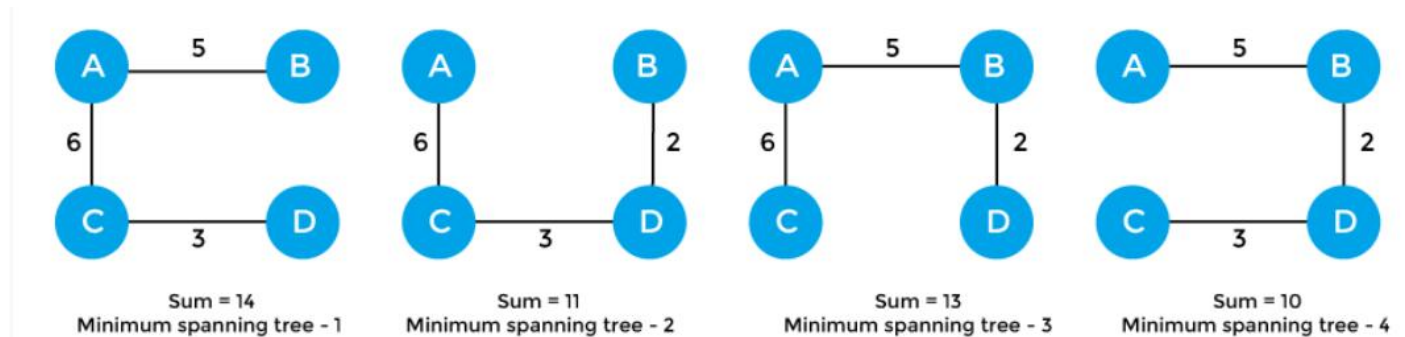
- A minimum spanning tree can be defined as the spanning tree in which the sum of the weights of the edge is minimum.
- The weight of the spanning tree is the sum of the weights given to the edges of the spanning tree.
- In the real world, this weight can be considered as the distance, traffic load, congestion, or any random value.

### Example of minimum spanning tree

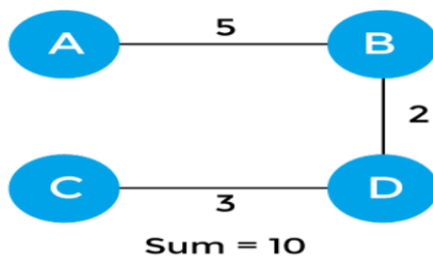
Let's understand the minimum spanning tree with the help of an example.



The sum of the edges of the above graph is 16. Now, some of the possible spanning trees created from the above graph are –



So, the minimum spanning tree that is selected from the above spanning trees for the given weighted graph is



### Real-World Application of Minimum Spanning Tree

- The building or Connecting of roads among cities or villages at a minimal cost.
- Network service providers for finding the minimal cost to provide service to every user.
- Cluster Analysis of data sets
- Protocols to avoid network cycles
- Maximum bottleneck paths
- A real-time face tracking algorithm
- Dithering algorithms
- Prime Factorial

### Algorithms for Minimum spanning tree

A minimum spanning tree can be found from a weighted graph by using the algorithms given below -

- Prim's Algorithm
- Kruskal's Algorithm

#### 4. Kruskal's Algorithm:

- Kruskal's algorithm is a realistic algorithm that provides the shortest path from one place to another in terms of edge weights or cost. This produces output exactly the same as Prim's algorithm but the process or algorithm differs.
- It greedily picks  $n-1$  edges one by one from the graph that has the minimum cost such that no cycle is created. So Kruskal's Algorithm takes up a connected and undirected graph and returns its minimum spanning tree.

#### Working Procedure:

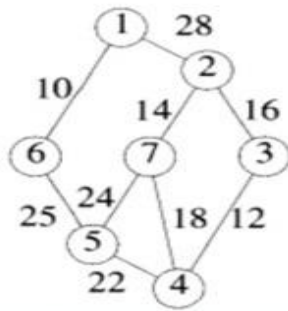
**The main idea is:** Always select the minimum-cost edge that does not form a cycle with the edges already selected.

To efficiently check whether an edge creates a cycle, we use the Disjoint Set (Union-Find) data structure.

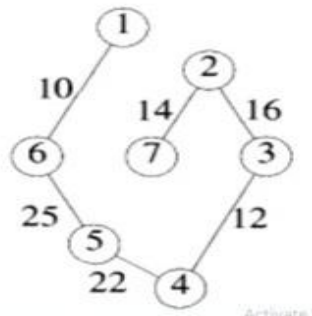
#### Procedure:

1. Store all edges of the graph along with their source, destination, and cost.
2. Create a Min-Heap / PriorityQueue containing all edges.  
The edge with the smallest cost is given the highest priority.
3. Initialize parent[].  
Initially, every vertex belongs to a separate set.  
Therefore:  $\text{parent}[i] = -1$
4. Remove the minimum-cost edge  $(u, v)$  from the PriorityQueue.
5. Find the set representatives of  $u$  and  $v$ :  
 $j = \text{Find}(u)$   
 $k = \text{Find}(v)$
6. Check for a cycle:  
If  $j == k$ , both vertices are already in the same set.  
Adding this edge would create a cycle.  
Therefore, reject the edge.  
If  $j \neq k$ , the edge does not create a cycle.  
Therefore, select the edge.
7. Add the selected edge to the MST and update the cost:  $\text{mincost} = \text{mincost} + \text{edge.cost}$
8. Union the two sets:  $\text{Union}(j, k)$
9. Repeat until  $n - 1$  edges are selected.
10. If  $n - 1$  edges are selected, the MST is obtained. Otherwise: No spanning tree

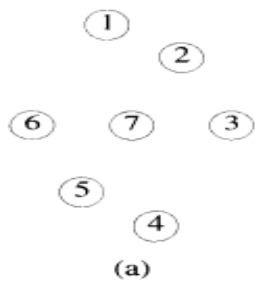
Example Graph:



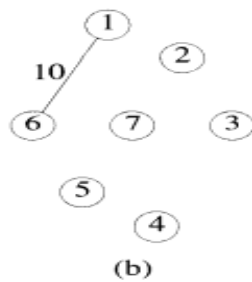
Graph



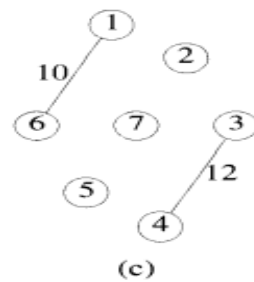
Resultant Graph



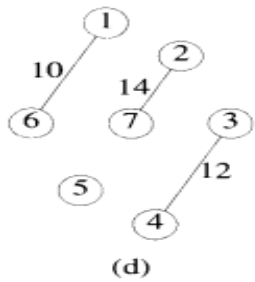
(a)



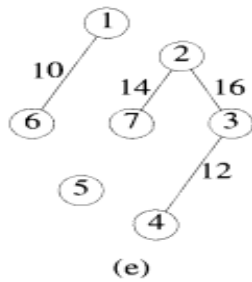
(b)



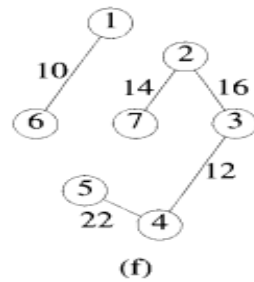
(c)



(d)



(e)



(f)

**Pseudo Code:**

```

1  Algorithm Kruskal( $E, cost, n, t$ )
2  //  $E$  is the set of edges in  $G$ .  $G$  has  $n$  vertices.  $cost[u, v]$  is the
3  // cost of edge  $(u, v)$ .  $t$  is the set of edges in the minimum-cost
4  // spanning tree. The final cost is returned.
5  {
6      Construct a heap out of the edge costs using Heapify;
7      for  $i := 1$  to  $n$  do  $parent[i] := -1$ ;
8      // Each vertex is in a different set.
9       $i := 0$ ;  $mincost := 0.0$ ;
10     while  $((i < n - 1)$  and  $(\text{heap not empty}))$  do
11     {
12         Delete a minimum cost edge  $(u, v)$  from the heap
13         and reheapify using Adjust;
14          $j := \text{Find}(u)$ ;  $k := \text{Find}(v)$ ;
15         if  $(j \neq k)$  then
16         {
17              $i := i + 1$ ;
18              $t[i, 1] := u$ ;  $t[i, 2] := v$ ;
19              $mincost := mincost + cost[u, v]$ ;
20             Union $(j, k)$ ;
21         }
22     }
23     if  $(i \neq n - 1)$  then write ("No spanning tree");
24     else return  $mincost$ ;
25 }
```

**Find with Path Compression**

```

Find(x)
{
    if  $parent[x] = -1$  then
        return  $x$ ;
    else
         $parent[x] := \text{Find}(parent[x])$ ; // Path compression
        return  $parent[x]$ ;
}
```

**Union by Rank / Simple Union**

```

Union(x, y)
{
     $xRoot := \text{Find}(x)$ ;
     $yRoot := \text{Find}(y)$ ;

    if  $(xRoot \neq yRoot)$  then
         $parent[yRoot] := xRoot$ ; // Merge  $yRoot$  into  $xRoot$ 
}
```

**Explanation:**

- **Find(x):** Returns the representative (root) of the set that contains  $x$ .
  - Path compression ensures that during recursive calls, all nodes point directly to the root, making future queries faster.
- **Union(x, y):** Merges the two sets containing  $x$  and  $y$ .
  - If their roots are different, one root becomes the parent of the other.

**Time Complexity:** With a PriorityQueue:

Creating heap  $\rightarrow O(E \log E)$

Find + Union  $\rightarrow$  approximately  $O(E \log V)$

Overall  $\rightarrow O(E \log E)$

Since  $E$  can be at most  $V^2$ , this is commonly written as:  $O(E \log E)$

**In one line:** Kruskal = Pick the cheapest edge  $\rightarrow$  check cycle using Find  $\rightarrow$  if no cycle, select it  $\rightarrow$  Union the sets  $\rightarrow$  repeat until  $n-1$  edges are selected.

**Java Program for Kruskal's algorithm:** **Kruskals\_Algo.java**

```
import java.util.*;

class Kruskal
{
    static class Edge
    {
        int u;
        int v;
        int cost;

        Edge(int u, int v, int cost)
        {
            this.u = u;
            this.v = v;
            this.cost = cost;
        }
    }

    // Find operation
    static int find(int parent[], int vertex) {
        if (parent[vertex] == -1)
            return vertex;

        return find(parent, parent[vertex]);
    }
}
```

```
// Union operation

static void union(int parent[], int root1, int root2)
{
    parent[root1] = root2;
}

// Kruskal's Algorithm
static void kruskal(Edge edges[], int n)
{
    // Create Min Heap of edges based on cost

    PriorityQueue<Edge> heap =
        new PriorityQueue<>(
            (a, b) -> Integer.compare(a.cost, b.cost)
        );

    // Insert all edges into heap
    for (Edge edge : edges)
    {
        heap.add(edge);
    }

    // Each vertex initially belongs to different set

    int parent[] = new int[n + 1];

    for (int i = 1; i <= n; i++) {
        parent[i] = -1;
    }

    // MST array

    Edge mst[] = new Edge[n - 1];

    int i = 0;
    int minCost = 0;

    // Kruskal main loop
    while (i < n - 1 && !heap.isEmpty()) {

        // Delete minimum-cost edge
        Edge edge = heap.poll();

        int u = edge.u;
        int v = edge.v;

        // Find the sets of u and v
        int j = find(parent, u);
        int k = find(parent, v);
```

```
// If they belong to different sets, include the edge
if (j != k) {

    i++;

    mst[i - 1] = edge;

    minCost = minCost + edge.cost;

    // Union the two sets
    union(parent, j, k);
}
}

// Check whether spanning tree exists
if (i != n - 1) {

    System.out.println("No spanning tree");
} else {

    System.out.println("\nMinimum Spanning Tree:");

    for (int j = 0; j < n - 1; j++) {

        System.out.println(
            mst[j].u + " - " +
            mst[j].v + " = " +
            mst[j].cost
        );
    }

    System.out.println(
        "\nMinimum Cost = " + minCost
    );
}
}

// Main Method

public static void main(String args[]) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of vertices: ");
    int n = sc.nextInt();

    System.out.print("Enter number of edges: ");
    int e = sc.nextInt();
}
```

```
Edge edges[] = new Edge[e];

System.out.println("Enter source destination cost:" );

// Read all edges
for (int i = 0; i < e; i++)
{
    int u = sc.nextInt();
    int v = sc.nextInt();
    int cost = sc.nextInt();
    edges[i] = new Edge(u, v, cost);
}

// Apply Kruskal's algorithm
kruskal(edges, n);

sc.close();
}
}
```

**input=**

Enter number of vertices: 7

Enter number of edges: 9

Enter source destination cost:

1 2 28

1 6 10

2 7 14

2 3 16

3 4 12

4 5 22

4 7 18

5 6 25

5 7 24

**output=**

Minimum Spanning Tree:

1 - 6 = 10

3 - 4 = 12

2 - 7 = 14

2 - 3 = 16

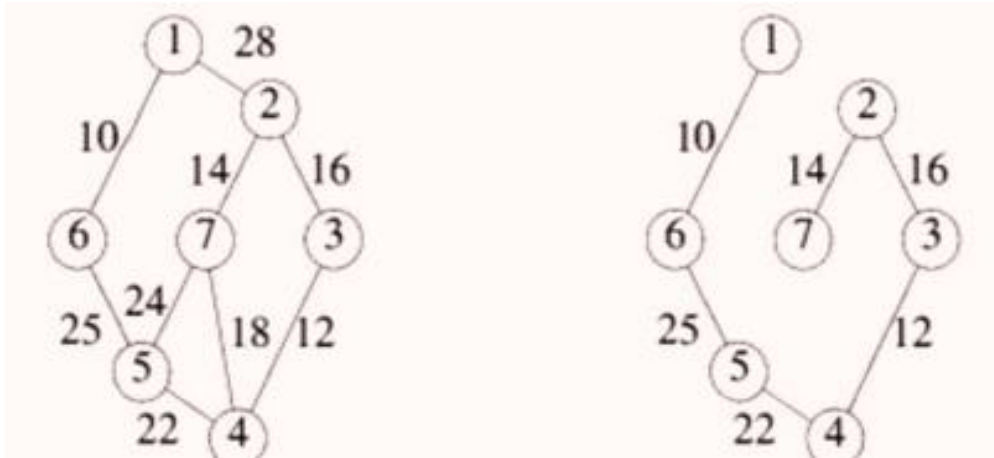
4 - 5 = 22

5 - 6 = 25

Minimum Cost = 99

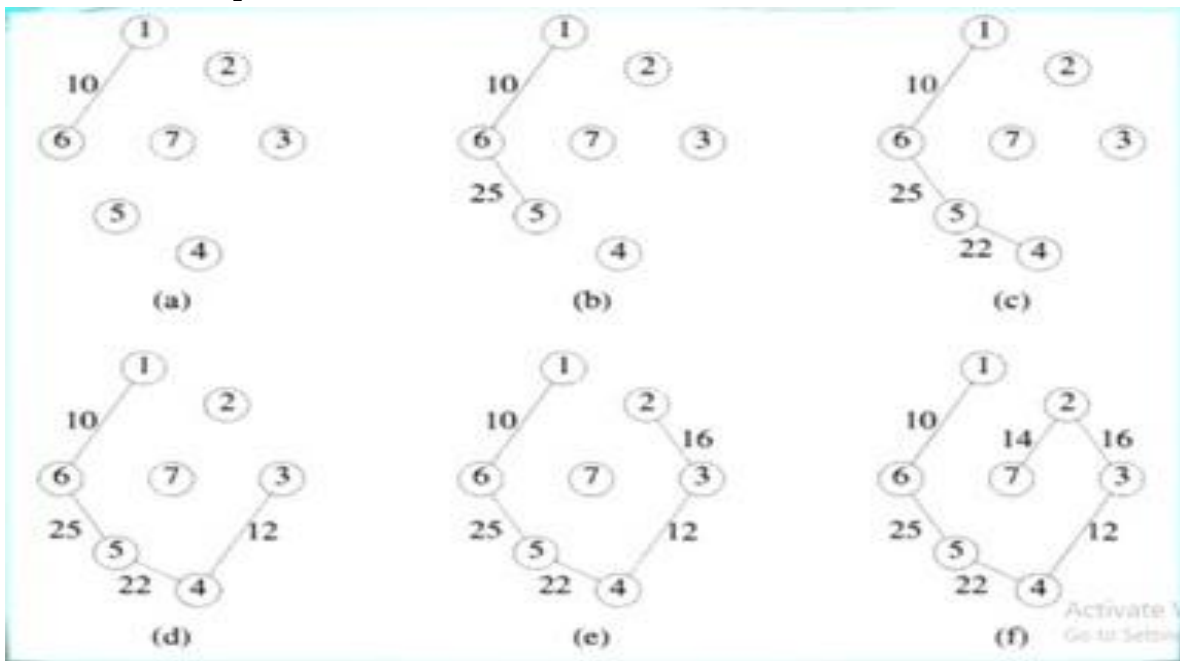
### 5. Prim's Algorithm:

- Prim's algorithm, in contrast with Kruskal's algorithm, treats the nodes as a single tree and keeps on adding new nodes to the spanning tree from the given graph.



Graph

Resultant Graph



### Main Idea of Prim's Algorithm:

1. Find the **minimum-cost edge** in the graph.
2. Include this edge in the MST.
3. Maintain near[] to identify the **nearest vertex already included in the MST** for every remaining vertex.
4. Among all vertices not yet included, select the one whose connection through near[] has the **minimum cost**.

5. Add that edge to the MST.
6. Update near[] because the newly selected vertex may provide a cheaper connection.
7. Repeat until exactly **n - 1 edges** are selected.

**Pseudo Code:**

```

1  Algorithm Prim(E, cost, n, t)
2  // E is the set of edges in G. cost[1 : n, 1 : n] is the cost
3  // adjacency matrix of an n vertex graph such that cost[i, j] is
4  // either a positive real number or  $\infty$  if no edge (i, j) exists.
5  // A minimum spanning tree is computed and stored as a set of
6  // edges in the array t[1 : n - 1, 1 : 2]. (t[i, 1], t[i, 2]) is an edge in
7  // the minimum-cost spanning tree. The final cost is returned.
8  {
9      Let (k, l) be an edge of minimum cost in E;
10     mincost := cost[k, l];
11     t[1, 1] := k; t[1, 2] := l;
12     for i := 1 to n do // Initialize near.
13         if (cost[i, l] < cost[i, k]) then near[i] := l;
14         else near[i] := k;
15     near[k] := near[l] := 0;
16     for i := 2 to n - 1 do
17     { // Find n - 2 additional edges for t.
18         Let j be an index such that near[j] ≠ 0 and
19         cost[j, near[j]] is minimum;
20         t[i, 1] := j; t[i, 2] := near[j];
21         mincost := mincost + cost[j, near[j]];
22         near[j] := 0;
23         for k := 1 to n do // Update near[ ].
24             if ((near[k] ≠ 0) and (cost[k, near[k]] > cost[k, j]))
25                 then near[k] := j;
26     }
27     return mincost;
28 }
```

**Time Complexity**

Depends on implementation:

1. **Using Adjacency Matrix + Simple Search** →  $O(V^2)$   
(Good for dense graphs).
  2. **Using Min-Heap + Adjacency List** →  $O(E \log V)$   
(Each extract-min takes  $O(\log V)$ , done  $V$  times, and each decrease-key for edges is  $O(\log V)$ ).
    - Extract-min:  $V$  times →  $O(V \log V)$
    - Decrease-key: up to  $E$  times →  $O(E \log V)$
- Adjacency list storage →  $O(V+E)$
  - Min-Heap for vertices →  $O(V)$

**Java Program for Prim's algorithm using Adjacency List:   Prims\_Algo.java**

```
import java.util.Scanner;

class Prims_Algo {

    // Prim's Algorithm
    static void prim(int[][] cost, int n) {

        // t[][] stores the edges of MST
        int[][] t = new int[n - 1][2];

        // near[i] stores the nearest vertex that is already included in MST
        int[] near = new int[n + 1];

        int mincost = 0;

        // Step 1: Find minimum-cost edge (k, l)

        int min = Integer.MAX_VALUE;
        int k = 0;
        int l = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = i + 1; j <= n; j++) {
                if (cost[i][j] < min) {
                    min = cost[i][j];
                    k = i;
                    l = j;
                }
            }
        }

        // Add first minimum-cost edge
        t[0][0] = k;
        t[0][1] = l;

        mincost = cost[k][l];

        // Step 2: Initialize near[]

        for (int i = 1; i <= n; i++) {
            if (cost[i][l] < cost[i][k])
                near[i] = l;
            else
                near[i] = k;
        }
    }
}
```

```
// k and l are already included in MST
near[k] = 0;
near[l] = 0;

// Step 3: Find remaining n-2 edges
for (int i = 1; i <= n - 2; i++) {

    int minEdge = Integer.MAX_VALUE;
    int j = 0;

    // Find vertex j such that
    // cost[j][near[j]] is minimum

    for (int m = 1; m <= n; m++) {

        if (near[m] != 0 &&
            cost[m][near[m]] < minEdge) {

            minEdge = cost[m][near[m]];
            j = m;
        }
    }

    // Add selected edge to MST
    t[i][0] = j;
    t[i][1] = near[j];

    // Add edge cost
    mincost = mincost + cost[j][near[j]];

    // j is now included in MST
    near[j] = 0;

    // Step 4: Update near[]

    for (int m = 1; m <= n; m++) {

        if (near[m] != 0 &&
            cost[m][near[m]] > cost[m][j]) {

            near[m] = j;
        }
    }
}

// Display MST

System.out.println("\n Minimum Spanning Tree:");

for (int i = 0; i < n - 1; i++) {
```

```

        int u = t[i][0];
        int v = t[i][1];

        System.out.println(
            u + " - " + v +
            " = " + cost[u][v]
        );
    }
    System.out.println("\nMinimum Cost = " + mincost);
}
// Main Method
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();

    int[][] cost = new int[n + 1][n + 1];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cost[i][j] = sc.nextInt();
        }
    }
    prim(cost, n);
    sc.close();
}
}

```

**input=**

7

0 28 999 999 999 10 999

28 0 16 999 999 999 14

999 16 0 12 999 999 999

999 999 12 0 22 999 18

999 999 999 22 0 25 24

10 999 999 999 25 0 999

999 14 999 18 24 999 0

**output=**

Minimum Spanning Tree:

1 - 6 = 10

5 - 6 = 25

4 - 5 = 22

3 - 4 = 12

2 - 3 = 16

7 - 2 = 14

Minimum Cost = 99

## 6. Single source shortest path Problem:

The **Single Source Shortest Path** (SSSP) problem is about finding the shortest path from a single source node to all other nodes in a weighted graph. The most famous algorithm to solve this problem using a **greedy** approach is **Dijkstra's Algorithm**.

### Dijkstra's Algorithm Overview:

Dijkstra's algorithm works for graphs with non-negative weights and follows a greedy approach by selecting the closest (smallest tentative distance) node that hasn't been processed yet. It keeps expanding the path from the source node, ensuring the shortest possible paths are calculated for all nodes.

### Steps:

#### 1. Initialization:

- Set the distance to the source node as 0 (since the distance from the source to itself is 0).
- Set the distance to all other nodes as infinity (since they are initially unreachable).
- Create a set of all unprocessed nodes.

#### 2. Greedy Selection:

- Choose the unprocessed node with the smallest known distance from the source. This node is considered processed (the shortest path to it is finalized).

#### 3. Relaxation:

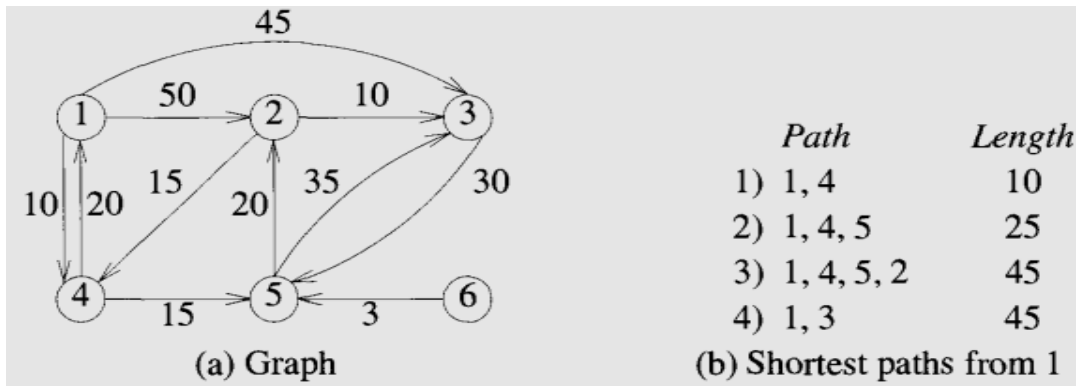
- For each neighboring node of the currently processed node, calculate its tentative distance from the source through the current node.
- If this new distance is smaller than the previously recorded distance, update the distance for the neighboring node.

#### 4. Repeat:

- Repeat steps 2 and 3 until all nodes have been processed or there are no more nodes with a finite distance.

#### 5. Result:

- After the algorithm finishes, you'll have the shortest path from the source to every other node.

**Pseudo Code:**

```

1  Algorithm ShortestPaths(v, cost, dist, n)
2  // dist[j],  $1 \leq j \leq n$ , is set to the length of the shortest
3  // path from vertex v to vertex j in a digraph G with n
4  // vertices. dist[v] is set to zero. G is represented by its
5  // cost adjacency matrix cost[1 : n, 1 : n].
6  {
7      for i := 1 to n do
8          { // Initialize S.
9              S[i] := false; dist[i] := cost[v, i];
10         }
11     S[v] := true; dist[v] := 0.0; // Put v in S.
12     for num := 2 to n - 1 do
13         {
14             // Determine n - 1 paths from v.
15             Choose u from among those vertices not
16             in S such that dist[u] is minimum;
17             S[u] := true; // Put u in S.
18             for (each w adjacent to u with S[w] = false) do
19                 // Update distances.
20                 if (dist[w] > dist[u] + cost[u, w]) then
21                     dist[w] := dist[u] + cost[u, w];
22         }
23     }

```

**Time Complexity:**

- Using a simple implementation with an array:  $O(V^2)$ , where *V* is the number of vertices.
- Using a priority queue (min-heap):  $O((V + E) \log V)$ , where *V* is the number of vertices and *E* is the number of edges.

**Java Program for Single source shortest path (Dijkstra's Algorithm): Dijkstra.java**

```
import java.util.Scanner;

class Dijkstra {

    static final int INF = 99999;

    // Function to find shortest paths
    static void shortestPaths(int[][] cost, int n, int source)
    {

        boolean[] visited = new boolean[n];
        int[] dist = new int[n];

        // Step 1: Initialize distance and visited arrays
        for (int i = 0; i < n; i++) {
            visited[i] = false;
            dist[i] = cost[source][i];
        }

        // Distance from source to itself is 0
        dist[source] = 0;
        visited[source] = true;

        // Step 2: Find shortest path for remaining vertices
        for (int count = 0; count < n - 1; count++) {

            int min = INF;
            int u = -1;

            // Find the unvisited vertex having minimum distance
            for (int i = 0; i < n; i++) {
                if (!visited[i] && dist[i] < min) {
                    min = dist[i];
                    u = i;
                }
            }

            // No reachable vertex found
            if (u == -1)
                break;

            // Mark the selected vertex as visited
            visited[u] = true;
        }
    }
}
```

```
// Update distances of adjacent vertices
for (int w = 0; w < n; w++) {

    if (!visited[w]
        && cost[u][w] != INF
        && dist[u] + cost[u][w] < dist[w]) {

        dist[w] = dist[u] + cost[u][w];
    }
}

// Print shortest distances
System.out.println("\nShortest Distances from Source Vertex " + (source + 1));

for (int i = 0; i < n; i++) {
    if (dist[i] == INF)
        System.out.println((source + 1) + " -> " + (i + 1) + " = INF");
    else
        System.out.println((source + 1) + " -> " + (i + 1) + " = " + dist[i]);
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    System.out.print("Enter number of vertices: ");
    int n = sc.nextInt();
    int[][] cost = new int[n][n];
    System.out.println("Enter the Cost Adjacency Matrix");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cost[i][j] = sc.nextInt();
        }
    }
    System.out.print("Enter Source Vertex (1 to " + n + "): ");
    int source = sc.nextInt();

    shortestPaths(cost, n, source - 1);

    sc.close();
}
```

Example:

input=

Enter number of vertices: 6

Enter the Cost Adjacency Matrix

0 50 45 10 99999 99999

99999 0 10 15 99999 99999

99999 99999 0 99999 30 99999

99999 99999 99999 0 15 99999

99999 20 35 99999 0 99999

99999 99999 99999 99999 99999 0

Enter Source Vertex (1 to 6): 1

output=

Shortest Distances from Source Vertex 1

1 -> 1 = 0

1 -> 2 = 45

1 -> 3 = 45

1 -> 4 = 10

1 -> 5 = 25

1 -> 6 = INF